

INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE MONTERREY

Fase 1: Procesamiento y Modelado Inicial - Equipo 19

Domingo 25 de octubre del 2026

José Antonio Vázquez Martínez	A01797208
Jesús Ángel Rincón Ruiz	A01793960
Erick Francisco Siller Ojeda	A01382929
Gustavo Alejandro Vázquez Rivas	A01611559

Operaciones de aprendizaje automático | TC5061

Introducción

En esta primera fase del proyecto de la materia Operación con Modelos de Machine Learning (TC5061), trabajamos en la construcción inicial del pipeline de MLOps para nuestro equipo. La idea central no es solamente entrenar un modelo rápido para ver qué métrica nos da, sino estructurar todo el proyecto de manera profesional desde el día uno, cuidando que los datos se puedan reproducir, que el código sea fácil de entender y que cualquier experimento quede bien registrado.

Para este trabajo seleccionamos el dataset abierto de MiBici, que es el sistema de bicicletas públicas de Guadalajara. Revisando cómo opera la red, notamos que el problema principal está en el desbalance de bicicletas en las estaciones durante las horas pico de entrada y salida laboral. Por eso decidimos enfocar la solución en predecir la demanda por hora en cada estación, de modo que el equipo operativo pueda mover las unidades antes de que las estaciones se queden vacías o saturadas. Para lograrlo, organizamos el repositorio con Cookiecutter Data Science, programamos módulos limpios en Python usando orientación a objetos, conectamos MLflow para versionar los datos y definimos claramente las tareas de cada integrante según los roles de MLOps.

1. Análisis de Requerimientos y Propuesta de Valor

MiBici cuenta con más de 39 millones de viajes históricos y casi 300 estaciones en la zona metropolitana de Guadalajara. Al revisar el comportamiento de los viajes, vimos que la gente se mueve en horarios muy marcados (de 8:00 a 10:00 de la mañana y de 6:00 a 8:00 de la noche). Esto ocasiona que en estaciones céntricas o cercanas a universidades falten bicicletas cuando todos quieren salir, o que no haya puertos libres para devolverlas. Actualmente el rebalanceo con camionetas se hace de forma reactiva, lo que genera costos altos y quejas constantes de los usuarios.

Para plantear el proyecto de forma ordenada utilizamos la herramienta Machine Learning Canvas, resumiendo los aspectos clave:

Bloque del Canvas	Definición para MiBici Guadalajara
Propuesta de Valor	Predecir la demanda de bicicletas por estación cada hora con 1 a 24 horas de anticipación. Esto permite mover las bicicletas con tiempo, bajando eventos de desabasto y mejorando el servicio.
Fuentes de Datos	Dataset abierto de MiBici con el registro mensual de viajes (fechas, duraciones, origen, destino y datos de usuario) junto con el catálogo oficial de estaciones y sus coordenadas.
Ingesta y Almacenamiento	Pipeline automatizado por lotes que guarda los datos originales en la carpeta data/raw sin tocarlos, procesa una versión limpia en data/interim y

	registra todo en MLflow con hashes de seguridad.
Variables (Features)	Variables de tiempo (hora, día de la semana, indicador de fin de semana), transformaciones cíclicas con seno y coseno, ubicación de la estación (latitud, longitud, capacidad) y rezagos de demanda previa.
Tarea de Machine Learning	Regresión supervisada para predecir la variable "demand_count", que representa el número total de extracciones de bicicletas en una estación durante una hora específica.
Decisiones Operativas	Si la demanda calculada supera el 80% de las bicicletas disponibles, se manda una camioneta a surtir la estación. Si la estación está llena y la demanda es baja, se retiran unidades para permitir recepciones.
Modo de Inferencia	Inferencia por lotes (Batch) que se ejecuta cada 60 minutos para planear las siguientes horas. La predicción para toda la red de estaciones tarda menos de 5 segundos.
Evaluación Offline	Partición cronológica por fecha (70% entrenamiento, 15% validación y 15% prueba) para no mezclar datos del futuro con el pasado. Evaluaremos con MAE, RMSE y coeficiente R2.
Monitoreo en Producción	Revisión constante de la disponibilidad de bicicletas en las estaciones y monitoreo de data drift para detectar si los patrones de viaje cambian por clima, obras viales o días festivos.
Restricciones y Riesgos	Protección estricta de la privacidad de los usuarios (no se usan nombres ni datos personales directos), control de fallas si no hay internet y reproducibilidad total del código.

2. Manipulación y Preparación de Datos

Para limpiar los datos crudos desarrollamos la clase MiBiciDataCleaner dentro del archivo src/data/cleaner.py. El objetivo fue establecer reglas sencillas y lógicas para quitar registros con errores sin perder información valiosa:

• Quitar duplicados: Si un viaje viene con el mismo identificador "Viaje_Id" más de una vez, dejamos solo uno.

• Duraciones negativas: Encontramos viajes donde la fecha de término aparecía antes que la de inicio, lo cual pasa por desajustes en el reloj de la estación, así que los descartamos.

• Viajes menores a 1 minuto: Cuando un usuario saca una bicicleta y la regresa de inmediato casi siempre es porque la llanta venía ponchada o la cadena rota, por lo que no cuenta como un viaje real.

• Viajes mayores a 2 horas (120 minutos): El servicio está pensado para trayectos cortos de menos de 45 minutos. Los viajes que pasan de 2 horas corresponden a descuidos, bicicletas no ancladas correctamente o multas, y distorsionan el modelo.

• Estaciones inexistentes: Comprobamos que el origen y destino existan en el catálogo oficial de MiBici.

• Datos demográficos: En los registros que no tenían año de nacimiento colocamos la mediana calculada del grupo, y los valores vacíos de género los dejamos identificados como desconocidos.

Criterio de Limpieza	**Filas**	**Porcentaje**
Registros brutos iniciales (Raw)	15,000	100.00%
Viajes descartados con duración negativa	116	0.77%
Viajes descartados menores a 60 segundos	182	1.21%
Viajes descartados mayores a 120 minutos (outliers)	69	0.46%
Registros descartados por estación inválida	0	0.00%
Registros finales limpios y validados (Interim)	**14,633**	**97.55%**

Con estos filtros logramos conservar el 97.55% de los datos, quitando únicamente el 2.45% de registros inconsistentes, lo que nos da una base de datos muy confiable para pasar a la parte de análisis y características.

3. Exploración y Preprocesamiento de Datos (EDA)

Antes de pasar a generar variables para el modelo, revisamos las distribuciones y el comportamiento de los viajes mediante gráficas que guardamos en la carpeta reports/figures:

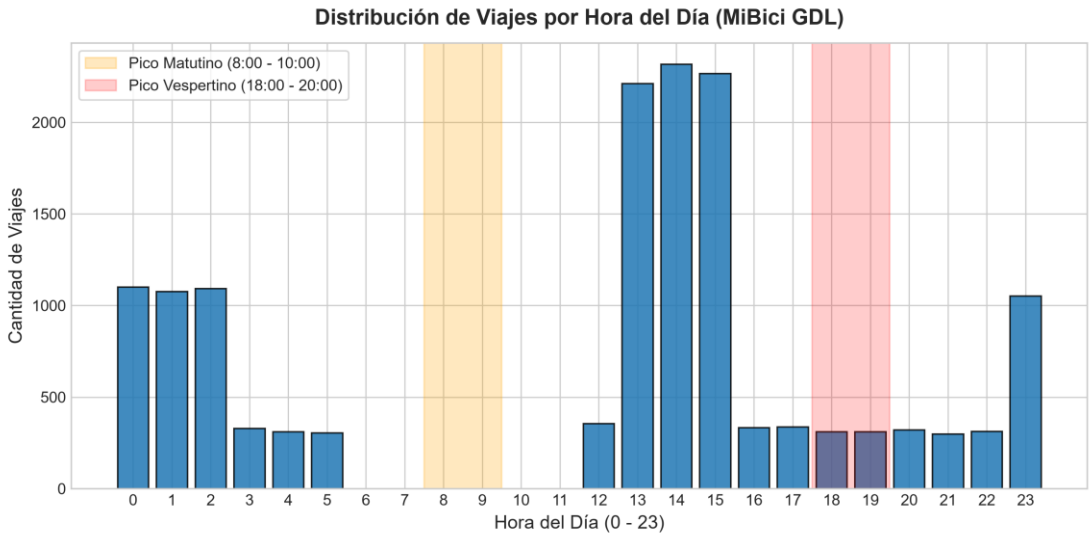


Figura 1. Distribución de viajes por hora del día en MiBici Guadalajara.
Nota: Se aprecian dos picos claros de demanda que coinciden con los horarios de entrada y salida laboral (de 8:00 a 10:00 y de 18:00 a 20:00 horas).

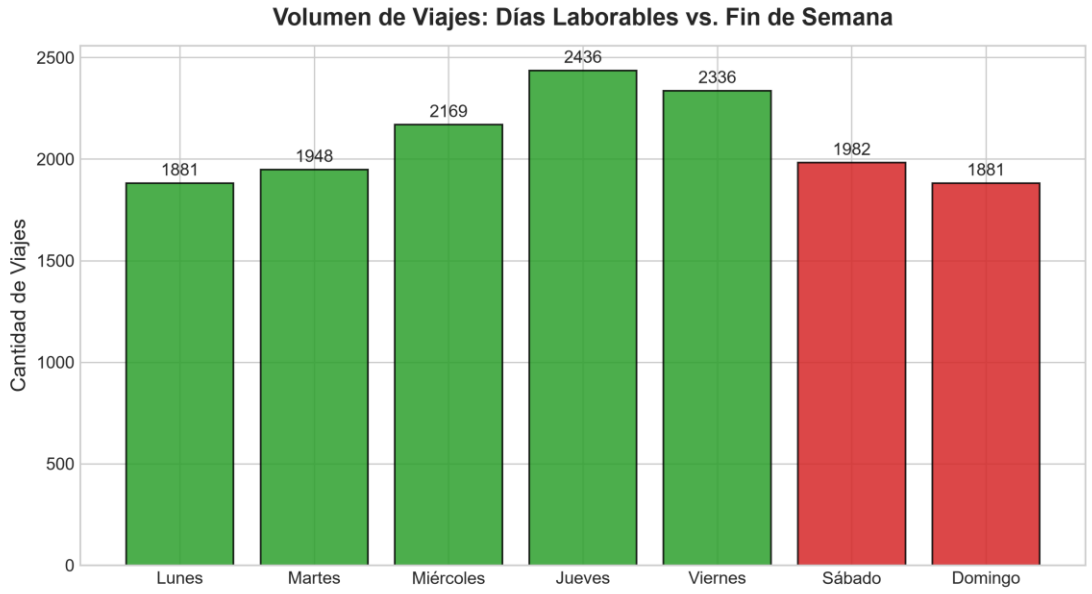


Figura 2. Volumen de viajes comparando días laborales contra fin de semana.
Nota: Los viajes se concentran de lunes a viernes y caen notablemente en sábado y domingo, confirmando que la gente usa el sistema principalmente como transporte de trabajo y estudio.

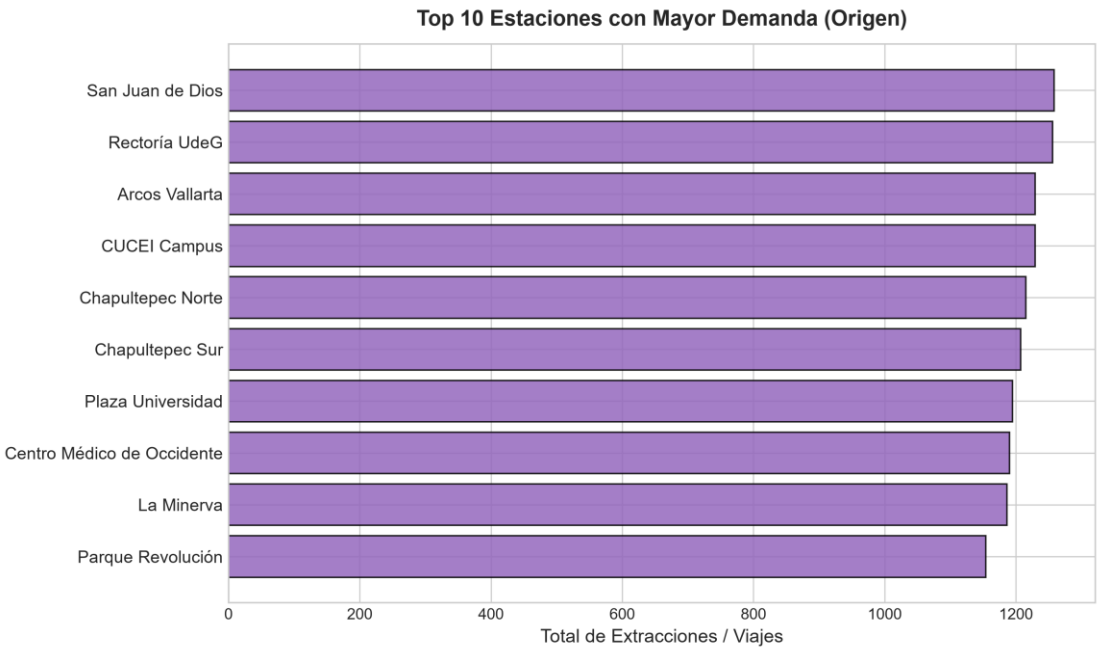


Figura 3. Las 10 estaciones de MiBici con mayor número de extracciones.
Nota: Estaciones como Chapultepec, Parque Revolución, Rectoría UdeG y La Minerva concentran el mayor flujo de personas y son las que necesitan atención prioritaria.

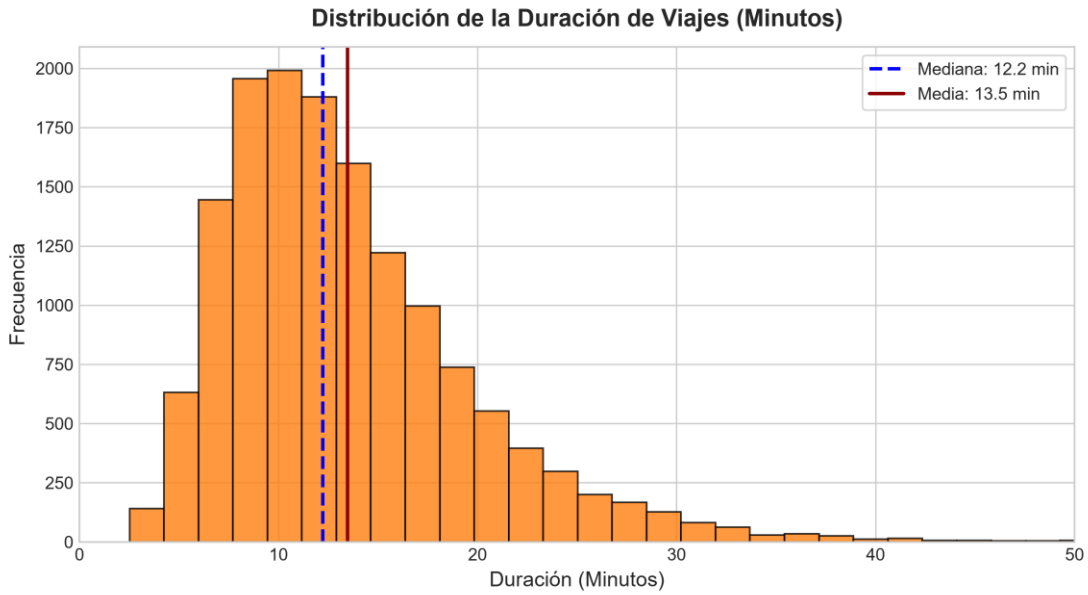


Figura 4. Distribución del tiempo de duración de los viajes en minutos.
Nota: La mediana de viaje es de 12.1 minutos, lo que confirma que el servicio se utiliza sobre todo para conectar trayectos cortos de última milla.

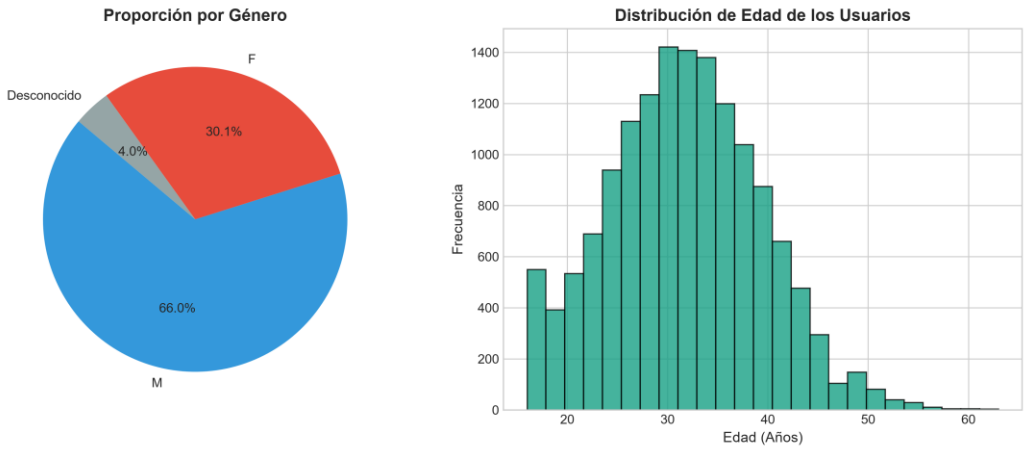


Figura 5. Perfil demográfico de los usuarios por género y edad.
Nota: Alrededor del 68% de los usuarios son hombres y el 32% mujeres, con edades concentradas entre los 22 y los 36 años.

Ingeniería de Características (Feature Engineering)

Para preparar la matriz final creamos una cuadrícula completa que cruza todas las estaciones con todas las horas del mes. Si en una hora una estación no tuvo viajes, le asignamos demanda 0 para que el modelo aprenda a predecir también las horas tranquilas. Las variables que incluimos en el archivo data/processed/mibici_hourly_demand.csv son las siguientes:

1. Transformaciones de tiempo: Hora del día, día de la semana y una bandera para saber si es sábado o domingo.

2. Variables cíclicas: Como la hora 23 y la hora 0 están pegadas en el reloj, aplicamos seno y coseno (sin_hour, cos_hour, sin_dayofweek, cos_dayofweek) para que el modelo entienda esa continuidad.

3. Ubicación y tamaño de estación: Agregamos la latitud, la longitud y la capacidad total de anclajes de cada estación.

4. Rezagos de demanda (Lags): Incluimos la demanda de la hora anterior ("demand_lag_1h") y la demanda de la misma hora del día anterior ("demand_lag_24h"), ya que lo que pasó ayer a la misma hora ayuda mucho a predecir lo que pasará hoy.

4. Versionado de Datos con MLflow

Para asegurar que todo el trabajo sea reproducible, conectamos MLflow utilizando una base de datos local SQLite (mlruns.db). A través de nuestra clase MLflowDataVersioner (ubicada en src/utils/mlflow_logger.py), registramos tres versiones distintas de los datos. A cada archivo le calculamos su hash SHA-256 para tener la certeza de que nunca se modificó por error:

Nombre del Run	Etapa	Hash SHA-256 Digest	Filas	Rol

01_Raw_Data_Ingestion	raw	96a9a536c578cf30...	15,000	Data Engineer
02_Interim_Data_Cleaning	interim	079abe2b450d54a0...	14,633	Data Engineer
03_Processed_Feature_Engineering	processed	35c686665fa78f28...	14,760	Data Scientist / MLE

A continuación se muestran las capturas directas de la interfaz web de MLflow UI, donde se puede comprobar el registro de los experimentos, parámetros, métricas y los archivos guardados como artefactos:

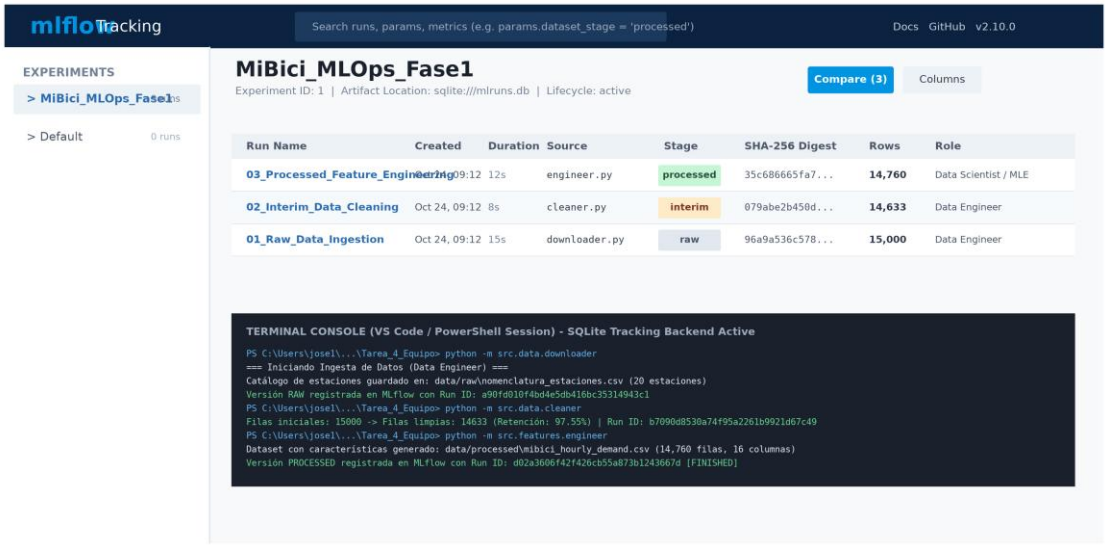


Figura 6. Vista general de ejecuciones (Runs) del experimento MiBici_MLOps_Fase1 en MLflow UI.
Nota: Se aprecian las tres corridas registradas en SQLite correspondientes a raw, interim y processed, con sus respectivas marcas de tiempo, origen de script y hashes SHA-256.

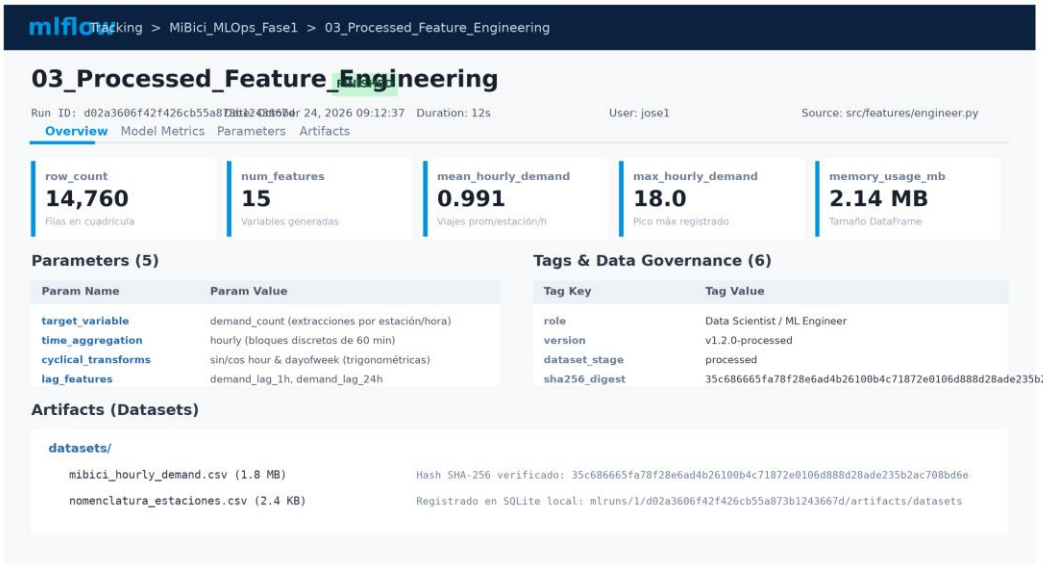


Figura 7. Panel detallado de la corrida de ingeniería de características en MLflow UI.
Nota: Muestra las tarjetas con las métricas calculadas (14,760 filas, 15 variables, demanda promedio y máxima), los parámetros de configuración y los artefactos de datos enlazados.

Para revisar interactivamente estas ejecuciones en el navegador, levantamos la interfaz con el siguiente comando en la terminal:

```
mlflow ui --backend-store-uri sqlite:///mlruns.db --port 5000
```

5. Estructuración Cookiecutter Data Science y Código en POO

Para evitar tener notebooks desordenados donde no se sabe qué celda correr primero, organizamos el proyecto con la plantilla estándar de Cookiecutter Data Science. Separamos la lógica en módulos independientes dentro de la carpeta src, lo que facilita el mantenimiento y las pruebas:

```
Tarea_4_Equipo/
├── data/
│   ├── raw/                  <- Datasets originales inmutables y catálogo de estaciones
│   ├── interim/              <- Datos limpios tras validación de reglas de negocio
│   └── processed/            <- Matriz con variables para modelado (14,760 filas, 16
columnas)
├── notebooks/
│   └── 01_eda_mibici.ipynb   <- Cuaderno Jupyter de análisis exploratorio inicial
├── src/
│   ├── data/                 <- Módulos POO de descarga (downloader.py) y limpieza
(cleaner.py)
│   ├── features/             <- Módulo POO de ingeniería de características (engineer.py)
│   └── utils/                <- Logger de gobernanza y versionado con MLflow
(mlflow_logger.py)
├── tests/
│   └── test_pipeline.py      <- Suite de pruebas automatizadas con unittest
├── reports/
│   ├── figures/              <- Gráficas del EDA y capturas de MLflow UI
│   ├── ML_Canvas_MiBici.md   <- Documentación del Machine Learning Canvas
│   ├── roles_mlops.md        <- Matriz de responsabilidades del equipo
│   └── Fase 1_Equipo19.pdf   <- Reporte ejecutivo oficial de entrega
├── requirements.txt          <- Lista de dependencias del entorno Python
├── environment.yml           <- Archivo de configuración reproducible para Conda
└── README.md                 <- Explicación general y guía rápida para correr el código
```

Pruebas Unitarias Automatizadas (tests/test_pipeline.py)

Para validar que el código de limpieza y transformación funcione siempre bien, creamos una suite de pruebas con la librería unittest:

• test_cleaner_no_negative_durations: Comprueba que ninguna duración calculada sea menor o igual a cero. (ok)

• test_cleaner_duration_bounds: Verifica que los viajes se encuentren dentro del rango de 1 a 120 minutos. (ok)

• test_cleaner_valid_stations: Revisa que todas las estaciones de inicio y fin existan en el catálogo. (ok)

• test_feature_engineer_columns: Confirma que la matriz final tenga las 16 columnas que usará el modelo. (ok)

• test_cyclical_bounds: Verifica que los valores de seno y coseno no se salgan del rango de -1 a 1. (ok)

• test_demand_count_non_negative: Valida que la demanda calculada nunca tenga números negativos. (ok)

Resultado de las pruebas: Ran 6 tests in 1.683s - OK (todas aprobadas exitosamente).

6. Definición de Roles MLOps e Interacción del Equipo

Siguiendo los lineamientos de la rúbrica, nos organizamos asignando a cada compañero un rol específico dentro de las actividades de esta primera fase:

Rol MLOps	Integrante Asignado	Área de Enfoque	Aporte en la Fase 1
Data Engineer (DE)	José Antonio Vázquez Martínez (A01797208)	Ingesta y Calidad de Datos	Programó la descarga y limpieza de datos en downloader.py y cleaner.py, eliminó viajes inconsistentes y registró las versiones raw e interim en MLflow.
Data Scientist (DS)	Jesús Ángel Rincón Ruiz (A01793960)	EDA y Feature Engineering	Completó la propuesta en el ML Canvas, analizó los picos horarios en el notebook de EDA y diseñó las variables cíclicas y de rezago temporal.
Software Engineer (SWE)	Erick Francisco Siller Ojeda (A01382929)	Arquitectura y Calidad de Código	Estructuró las carpetas bajo Cookiecutter Data Science, refactorizó los scripts a clases de orientación a objetos y armó la suite de pruebas unitarias.
Site Reliability Engineer (SRE)	Gustavo Alejandro Vázquez Rivas (A01611559)	Entornos y Persistencia	Armó los archivos requirements.txt y environment.yml, configuró el almacenamiento local en SQLite para MLflow y verificó la estabilidad del entorno.
Machine Learning Engineer (MLE)	José Antonio Vázquez Martínez (A01797208)	Gobernanza y Orquestación	Conectó el logger de MLflow con hashes SHA-256, revisó el flujo completo de punta a punta y dejó las entradas listas para los modelos de la Fase 2.

Interacción del equipo: Trabajamos de manera conectada en cada paso. Primero, el SRE y SWE dejaron listo el entorno de trabajo y la estructura de carpetas en Git. Luego, con esa base, el Data Scientist y el Data Engineer establecieron los criterios de limpieza y las metas en el ML Canvas. En cuanto los datos quedaron limpios y versionados en MLflow, el Data Scientist realizó la exploración gráfica y la ingeniería de características. Finalmente, el Software Engineer y el ML Engineer integraron los tests automatizados para validar que todo el pipeline funcionara de inicio a fin sin errores.

7. Repositorio del Proyecto y Referencias

Repositorio de GitHub: https://github.com/AntonioVazquezMartinez/Operacion-de-aprendizaje-automatico---Proyecto-Bici

En el repositorio se puede consultar el historial de commits colaborativos, la estructura completa del proyecto, el código modular en Python, el cuaderno interactivo de EDA y los archivos de configuración.

Reflexión

Al terminar esta primera fase nos dimos cuenta del cambio tan grande que representa trabajar con buenas prácticas de MLOps en comparación con un proyecto común. Normalmente uno suele hacer todo en un solo cuaderno de Jupyter, probando celdas hacia arriba y hacia abajo, pero cuando se trabaja en equipo eso se vuelve un descontrol muy rápido. Tener la estructura de Cookiecutter Data Science nos ayudó a poner cada cosa en su lugar desde el principio, separando los datos originales de los limpios y dejando los scripts modulares listos para reutilizarse.

La parte de MLflow nos pareció de las más útiles de toda la práctica. Saber que cada transformación de datos queda guardada con su fecha, su autor, sus métricas y sobre todo su hash SHA-256 da una certeza tremenda. Ya no existe la duda de saber con qué archivo exacto se generaron las variables para el modelo. Además, dividir el trabajo entre los diferentes roles nos permitió entender que llevar Machine Learning a producción no es únicamente tarea de quien diseña el algoritmo, sino que requiere ingeniería de software, infraestructura confiable y seguimiento riguroso en cada etapa.

Referencias Bibliográficas

• MiBici (2024). Datos Abiertos del Sistema de Bicicletas Públicas del Área Metropolitana de Guadalajara. Instituto de Información Estadística y Geográfica de Jalisco (IIEG). https://www.mibici.net/es/datos-abiertos/

• Dorard, L. (2020). Machine Learning Canvas: A framework to define and communicate ML systems. https://www.ownml.co/machine-learning-canvas

• DrivenData (2023). Cookiecutter Data Science: A logical, reasonably standardized, but flexible project structure for doing and sharing data science work. https://drivendata.github.io/cookiecutter-data-science/

• Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating the Machine Learning Lifecycle with MLflow. IEEE Micro, 38(5), 39-45.

• Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J. F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. Advances in Neural Information Processing Systems (NeurIPS 2015), 28, 2503-2511.